LAPORAN TUGAS KELOMPOK

Mata Kuliah Sistem Operasi (USA-WP2260110)

Implementasi Komunikasi Antarproses (IPC) Pengiriman Data Sensor ke Proses Logger pada Kebun Pintar

Keterangan	Isi
Kode dan judul tugas	K-1 Implementasi Komunikasi Antarproses (IPC)
Kelas / tema	IF-3C — IoT dan Smart Farming
Nama	Dendy Zainal Mutaqien
NIM	250660121068
Anggota kelompok	Dendy Zainal Mutaqien (rincian di Lampiran D)
Nama host	INFINIX-ElementaryOS
Minggu / tanggal kumpul	Minggu ke-4 / 29 Oktober 2026
Kepingan puzzle	Kepingan #4 → Level 2 Puzzle UTS
Dosen pengampu	Muhammad Agreindra Helmiawan, S.Kom., M.T

Program Studi S1 Informatika
Fakultas Teknologi Informasi · Universitas Sebelas April
2026

Daftar Isi

1. Pendahuluan **3**
1.1 Latar belakang kasus 3
1.2 Tujuan tugas 3
1.3 Ruang lingkup 3
2. Dasar Teori Singkat **4**
3. Lingkungan Praktik **4**
4. Langkah Pengerjaan **5**
4.1 Langkah 1: Menyiapkan lingkungan dan broker 5
4.2 Langkah 2: Menyeragamkan format pesan 6
4.3 Langkah 3: Implementasi pipe 6
4.4 Langkah 4: Implementasi message queue 8
4.5 Langkah 5: Implementasi publish-subscribe dengan MQTT 10
4.6 Langkah 6: Pengukuran dan pemeriksaan 11
5. Hasil dan Analisis **13**
5.1 Hasil 13
5.2 Analisis 15
5.3 Rekomendasi 16
6. Kesimpulan **16**
7. Kepingan Puzzle **16**
Daftar Pustaka **16**
Lampiran A. Kode Program dan Log **18**
Lampiran B. Pernyataan Penggunaan AI Generatif **25**
Lampiran C. Daftar Periksa Sebelum Mengumpulkan **25**
Swa-nilai berdasarkan kriteria lembar tugas 25
Lampiran D. Lembar Kontribusi Anggota **25**

1. Pendahuluan

1.1 Latar belakang kasus

Kebun pintar memakai beberapa sensor yang membaca kondisi lahan secara terus menerus: kelembapan tanah, suhu udara, dan debit air pompa irigasi. Setiap sensor menghasilkan angka baru dalam hitungan detik, dan angka itu harus sampai ke satu proses pencatat (logger) yang menyimpannya sebagai riwayat. Kalau pembacaan tidak tercatat rapi, keputusan penyiraman otomatis kehilangan dasarnya: pompa bisa menyala saat tanah sudah basah, atau mati saat tanaman mulai kering.

Persoalannya bukan sekadar memindahkan angka. Sensor dan logger adalah proses terpisah, dan proses di sistem operasi tidak berbagi ruang alamat. Ketiganya berjalan bersamaan, sehingga dua sensor bisa menulis pada saat yang hampir sama. Mekanisme komunikasi antarproses (interprocess communication, IPC) yang dipilih menentukan apakah rekaman tetap utuh, apakah ada yang hilang, apakah pembacaan darurat bisa didahulukan, dan berapa besar biaya waktunya. Tugas ini menguji tiga mekanisme tersebut pada kasus yang sama, lalu membandingkannya secara terukur.

1.2 Tujuan tugas

Capaian pembelajaran yang diukur adalah Sub-CPMK-4 (CPMK061, CPMK063, CPMK081, CPMK102, CPMK112). Dijabarkan ke dalam tujuan berikut:

- **Pertama,** membangun jalur pengiriman data dari tiga proses sensor ke satu proses logger memakai pipe dan message queue, lalu menambahkan pola publish-subscribe dengan MQTT;
- **Kedua,** menguji setiap mekanisme dan mendokumentasikan perilakunya: jumlah rekaman diterima, rekaman hilang, rekaman rusak, dan latensi kirim ke terima;
- **Ketiga,** menganalisis kelebihan dan keterbatasan masing-masing mekanisme untuk kasus kebun pintar, serta memilih yang paling sesuai dengan kebutuhan kelas.

1.3 Ruang lingkup

Pekerjaan dibatasi pada hal berikut.

- Sistem yang diamati adalah tiga proses sensor dan satu proses logger pada satu mesin Linux, bukan perangkat keras sensor sesungguhnya. Nilai sensor dibangkitkan secara sintetis dengan pola yang sama seperti pembacaan nyata.
- Mekanisme yang diimplementasikan adalah message passing: pipe anonim, POSIX message queue, dan MQTT. Shared memory tidak diimplementasikan karena lembar tugas memberi pilihan salah satu, dan perbandingan yang diminta sudah terpenuhi oleh tiga mekanisme message passing.
- Pengukuran latensi dilakukan pada satu host, sehingga angkanya menunjukkan biaya relatif antar mekanisme, bukan latensi jaringan produksi.
- Yang tidak dikerjakan: antarmuka pengguna, penyimpanan ke basis data, dan pengamanan kanal MQTT (TLS maupun autentikasi).

2. Dasar Teori Singkat

Proses yang berjalan terpisah memiliki ruang alamat masing-masing, sehingga bertukar data menuntut bantuan kernel. Silberschatz dkk. (2018) membagi mekanisme tersebut menjadi dua keluarga besar: shared memory, tempat dua proses memetakan satu region memori yang sama, dan message passing, tempat proses saling mengirim pesan lewat fasilitas kernel. Shared memory lebih cepat setelah pemetaan selesai, tetapi sinkronisasinya sepenuhnya menjadi tanggung jawab program. Message passing memindahkan beban salin dan pengaturan antrean ke kernel, dengan imbalan biaya per pesan yang lebih besar.

Pipe adalah bentuk message passing paling sederhana, dijelaskan Kerrisk (2010) sebagai saluran byte satu arah antara dua proses yang memiliki hubungan kekerabatan, biasanya dibuat dengan fork. Karena pipe adalah byte stream, ia tidak mengenal batas pesan: seratus byte yang ditulis bisa terbaca sebagai satu potongan atau beberapa potongan. Kernel hanya menjamin bahwa penulisan sebesar PIPE_BUF byte atau kurang bersifat atomik, artinya tidak tersela penulis lain. Pemisahan rekaman karena itu harus dilakukan sendiri oleh penerima, misalnya memakai karakter pemisah dan menyimpan sisa byte yang belum membentuk rekaman utuh.

POSIX message queue memperbaiki kelemahan itu. Setiap pesan disimpan sebagai unit tersendiri dengan panjang dan prioritas, sehingga mq_receive selalu mengembalikan tepat satu pesan utuh. Antrean punya nama di /dev/mqueue, sehingga proses yang tidak berkerabat dapat ikut mengirim. Pesan berprioritas lebih tinggi dilayani lebih dulu, dan pesan yang melebihi mq_msgsize ditolak dengan EMSGSIZE, bukan dipotong diam-diam. Antrean juga bertahan selama masih ada yang membukanya atau selama belum dihapus dengan mq_unlink.

Publish-subscribe menambah satu pihak di tengah: broker. Pengirim (publisher) tidak mengetahui siapa penerimanya; ia hanya menempelkan pesan pada sebuah topik, dan broker meneruskannya ke semua pelanggan topik itu (OASIS, 2019). Karena pengiriman tidak langsung, ketahanan pengiriman diatur lewat tingkat kualitas layanan (QoS). Pada QoS 1, broker boleh mengirim ulang pesan yang belum dikonfirmasi, sehingga penerima dapat menerima rekaman yang sama lebih dari sekali dan perlu bersikap idempoten. Sebaliknya, sifat fan-out membuat beberapa logger bisa menerima aliran data yang sama tanpa saling berebut, sesuatu yang tidak diberikan satu antrean pipe maupun message queue.

3. Lingkungan Praktik

Komponen	Keterangan
Laptop (host)	elementary OS 8, AMD Ryzen 7 7735HS, RAM 14 GiB, kernel 7.0.0-34-generic
Hypervisor	Tidak dipakai — seluruh percobaan dijalankan langsung pada host
VM / host	INFINIX-ElementaryOS, 16 vCPU, tanpa virtualisasi
Perangkat lain	Tidak berlaku — nilai sensor dibangkitkan secara sintetis, tanpa Raspberry Pi
Bahasa dan alat	Python 3.12.3; paho-mqtt 2.1.0; broker amqtt 0.12.1; ctypes untuk memanggil libc mq_open/mq_send/mq_receive; python-docx dan matplotlib untuk laporan

Komponen	Keterangan
Batas kernel	PIPE_BUF = 4096 byte; /proc/sys/fs/mqueue/msg_max = 10 pesan; msgsize_max = 8192 byte

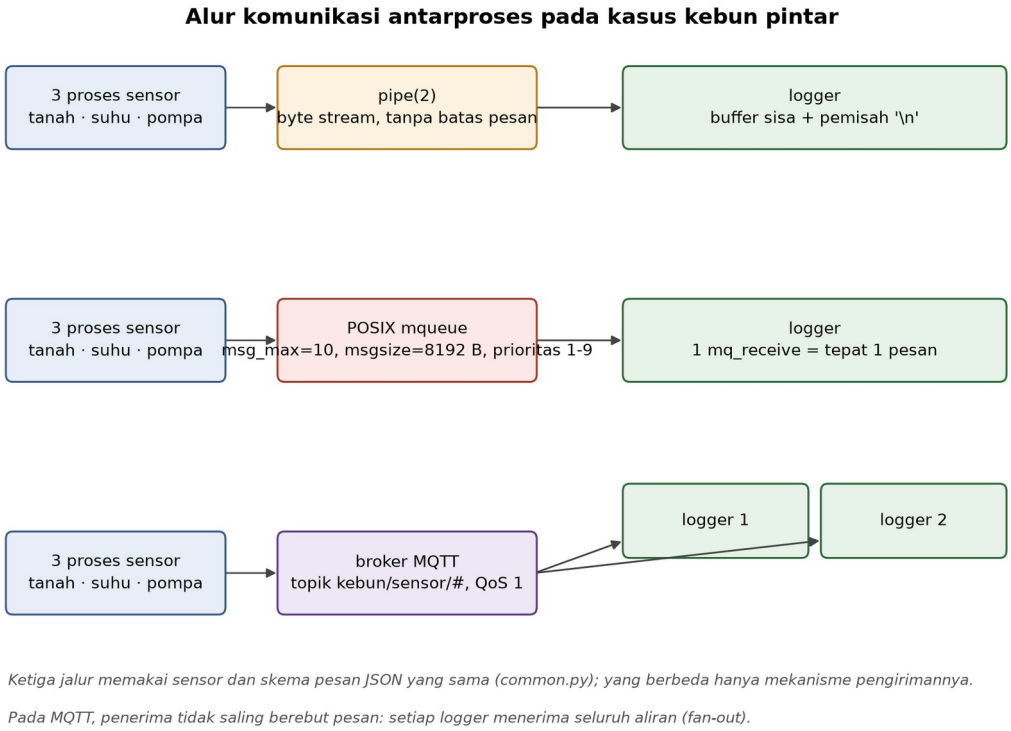
Broker MQTT memakai amqtt yang dipasang di virtual environment proyek, bukan mosquitto, karena paket mosquitto tidak tersedia di mesin dan tidak ada akses sudo untuk memasangnya. Pola publish-subscribe yang diuji tetap sama: topik, QoS, dan fan-out ke beberapa pelanggan.

4. Langkah Pengerjaan

Seluruh kode berada dalam satu folder proyek dengan struktur berikut.

```
ipc-kebun/
├── common.py          format pesan JSON dan penulisan CSV
├── mq.py              pembungkus POSIX mq_* lewat ctypes
├── pipe_demo.py       tiga proses sensor → satu pipe
├── mqueue_demo.py     tiga proses sensor → satu message queue
├── mqtt_demo.py       publish-subscribe lewat broker
├── bench.py           pengukuran latensi dan kehilangan data
├── test_basic.py      pemeriksaan batas pesan dan prioritas
├── figures.py         diagram alur dan grafik latensi
├── broker.yaml        konfigurasi broker amqtt
├── run_all.sh         menjalankan seluruh percobaan
└── results/           keluaran CSV, JSON, dan gambar
```

Gambar 4.1 memperlihatkan ketiga jalur komunikasi yang dibangun. Ketiganya berangkat dari proses sensor yang sama dan berakhir di proses logger; yang berbeda adalah perantara yang membawa pesan.



Gambar 4.1 Diagram alur komunikasi ketiga mekanisme IPC

4.1 Langkah 1: Menyiapkan lingkungan dan broker

Percobaan MQTT memerlukan broker yang mendengarkan di port 1883. Karena mosquitto tidak tersedia, amqtt dipakai sebagai broker pengganti dan dijalankan di latar belakang. Sisanya cukup pustaka standar Python, ditambah paho-mqtt untuk sisi klien.

```
$ python3 -m venv .venv
$ .venv/bin/pip install paho-mqtt amqtt
$ .venv/bin/amqtt -c broker.yaml &
[INFO] Listener 'default' bind to 127.0.0.1:1883 (max_connections=0)
$ ss -ltn | grep 1883
LISTEN 0 100 127.0.0.1:1883 0.0.0.0:*
```

Broker hanya mendengarkan alamat lokal karena percobaan tidak memerlukan akses dari mesin lain. Konfigurasi broker.yaml mematikan autentikasi dan pemeriksaan topik agar fokus tetap pada mekanisme IPC.

4.2 Langkah 2: Menyeragamkan format pesan

Ketiga mekanisme diuji dengan beban yang sama, sehingga format pesannya harus identik. Setiap rekaman berisi nama sensor, nilai pembacaan, nomor urut, dan waktu pengambilan; nomor urut dipakai untuk mendeteksi rekaman yang hilang atau terduplikasi setelah transfer.

```
"""Utilitas bersama: format pesan sensor dan penyimpanan ke CSV."""
import json
import os

SENSORS = {
    "tanah": ("kelembapan", "%"),
    "suhu": ("celsius", "C"),
    "pompa": ("debit", "L/m"),
}

QUEUE_PIPE = "/tmp/ipc-kebun-pipe"
QUEUE_MQ = "/kebun-sensor"
LOG_PIPE = "results/log-pipe.csv"
LOG_MQ = "results/log-mq.csv"
LOG_MQTT = "results/log-mqtt.csv"


def buat_pesan(sensor, nilai, seq, ts):
    """Satu pembacaan sensor sebagai bytes JSON, diakhiri newline."""
    return (json.dumps({"sensor": sensor, "nilai": nilai, "seq": seq, "ts": ts}) + "\
n").encode()


def tulis_csv(path, baris, ada_header=None):
    """Tambah satu baris ke CSV. Header ditulis kalau berkas belum ada."""
    baru = ada_header if ada_header is not None else not os.path.exists(path)
    with open(path, "a", encoding="utf-8") as f:
        if baru:
            f.write("recv_ts,sensor,nilai,seq,ts\n")
        f.write(",".join(str(x) for x in baris) + "\n")
```

Fungsi buat_pesan mengembalikan satu baris JSON yang diakhiri newline. Penanda newline itu penting untuk pipe, yang tidak mengerti batas pesan; untuk message queue dan MQTT penanda tersebut hanya mempermudah pembacaan berkas log.

4.3 Langkah 3: Implementasi pipe

Tiga proses sensor dibuat dengan fork dan mewarisi ujung tulis pipe. Proses induk menutup ujung tulisnya sendiri, lalu membaca sampai jumlah rekaman yang diterima mencapai jumlah yang dikirim. Karena pipe hanya byte stream, penerima memecah potongan baca berdasarkan newline dan menyimpan sisa byte yang belum utuh pada variabel sisa untuk digabungkan dengan potongan berikutnya.

```
"""Pipe: beberapa proses sensor menulis ke satu pipe, logger membaca.
```

```
Pipe adalah byte stream tanpa batas pesan. Logika pemisah rekaman ada di sini
(newline), bukan di kernel. Jalankan --tanpa-delimiter untuk melihat akibatnya.
"""
import argparse
import json
import os
import sys
import time

import common

PIPE_BUF = 4096


def sensor(nama, n, fd_tulis, pakai_delimiter):
    for seq in range(n):
        pesan = common.buat_pesan(nama, round(20 + seq * 0.5, 2), seq, time.time())
        if not pakai_delimiter:
            pesan = pesan.rstrip(b"\n")
        # os.write menulis utuh selama <= PIPE_BUF; di atas itu bisa disela proses lain.
        os.write(fd_tulis, pesan)
        time.sleep(0.002)
    os.close(fd_tulis)


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--sensors", type=int, default=3)
    p.add_argument("--n", type=int, default=20)
    p.add_argument("--tanpa-delimiter", action="store_true")
    a = p.parse_args()

    os.makedirs("results", exist_ok=True)
    baca, tulis = os.pipe()
    nama_sensor = list(common.SENSORS)[: a.sensors]
    pakai_delimiter = not a.tanpa_delimiter

    for nama in nama_sensor:
        if os.fork() == 0:
            os.close(baca)
            sensor(nama, a.n, tulis, pakai_delimiter)
            os._exit(0)
    os.close(tulis)

    target = a.n * len(nama_sensor)
    diterima = 0
    rusak = 0
    gabung = 0
    sisa = b""
    while diterima < target:
        chunk = os.read(baca, PIPE_BUF)
        if not chunk:
            break
        if chunk.count(b"{") > 1:
            gabung += 1
        if not pakai_delimiter:
            # Tidak ada cara memulihkan batas pesan setelah rekaman menempel.
            try:
                pesan = json.loads(chunk)
            except json.JSONDecodeError:
                rusak += 1
                continue
            common.tulis_csv(common.LOG_PIPE, (round(time.time(), 4), pesan["sensor"],
                                               pesan["nilai"], pesan["seq"], pesan["ts"]))
            diterima += 1
            continue
        buf = sisa + chunk
        baris = buf.split(b"\n")
        sisa = baris.pop()
        for b in baris:
            pesan = json.loads(b)
            common.tulis_csv(common.LOG_PIPE, (round(time.time(), 4), pesan["sensor"],
                                               pesan["nilai"], pesan["seq"], pesan["ts"]))
            diterima += 1
```

```
    while True:
        try:
            os.wait()
        except ChildProcessError:
            break

    print(f"pipe: diterima={diterima}/{target} rusak={rusak} read_multipesan={gabung}")
    if gabung:
        print(f"  {gabung} read berisi lebih dari satu rekaman: pipe tidak menjaga batas
pesan")


if __name__ == "__main__":
    main()
```

Untuk menunjukkan bahwa batas pesan memang tidak dijaga kernel, program juga menerima opsi --tanpa-delimiter yang mengirim JSON tanpa newline. Pada mode ini penerima mencoba membaca setiap potongan baca sebagai satu rekaman, sehingga rekaman yang menempel akan gagal diurai dan dihitung sebagai rusak.

4.4 Langkah 4: Implementasi message queue

Python tidak menyediakan POSIX message queue di pustaka standarnya. Alih-alih memasang paket pihak ketiga, kelima fungsi yang dibutuhkan dipanggil langsung dari libc lewat ctypes. Pembungkus mq.py di bawah ini memetakan mq_open, mq_send, mq_receive, mq_close, dan mq_unlink beserta struktur atribut antrean.

```
"""Pembungkus POSIX message queue lewat ctypes, tanpa paket tambahan."""
import ctypes
import os

_libc = ctypes.CDLL("libc.so.6", use_errno=True)
_libc.mq_open.argtypes = [ctypes.c_char_p, ctypes.c_int, ctypes.c_uint, ctypes.c_void_p]
_libc.mq_open.restype = ctypes.c_int
_libc.mq_send.argtypes = [ctypes.c_int, ctypes.c_char_p, ctypes.c_size_t, ctypes.c_uint]
_libc.mq_send.restype = ctypes.c_int
_libc.mq_receive.argtypes = [ctypes.c_int, ctypes.c_char_p, ctypes.c_size_t,
ctypes.POINTER(ctypes.c_uint)]
_libc.mq_receive.restype = ctypes.c_ssize_t
_libc.mq_close.argtypes = [ctypes.c_int]
_libc.mq_unlink.argtypes = [ctypes.c_char_p]

O_RDONLY, O_WRONLY, O_CREAT, O_EXCL = 0, 1, 0o100, 0o200


class MQAttr(ctypes.Structure):
    _fields_ = [
        ("mq_flags", ctypes.c_long),
        ("mq_maxmsg", ctypes.c_long),
        ("mq_msgsize", ctypes.c_long),
        ("mq_curmsgs", ctypes.c_long),
    ]


class MQError(OSError):
    pass


def _cek(hasil, nama):
    if hasil < 0:
        e = ctypes.get_errno()
        raise MQError(e, f"mq_{nama}: {os.strerror(e)}")
    return hasil


def buka(nama, mode, maks_pesan=10, ukuran_pesan=1024, buat=False, eksklusif=False):
    flag = O_CREAT if buat else 0
    if eksklusif:
        flag |= O_EXCL
    attr = MQAttr(0, maks_pesan, ukuran_pesan, 0) if buat else None
```

```
    ptr = ctypes.byref(attr) if attr is not None else None
    fd = _cek(_libc.mq_open(nama.encode(), mode | flag, 0o600, ptr), "open")
    return fd


def kirim(fd, data, prioritas=0):
    _cek(_libc.mq_send(fd, data, len(data), prioritas), "send")


def terima(fd, ukuran_pesan=1024):
    buf = ctypes.create_string_buffer(ukuran_pesan)
    prio = ctypes.c_uint(0)
    n = _cek(_libc.mq_receive(fd, buf, ukuran_pesan, ctypes.byref(prio)), "receive")
    return buf.raw[:n], prio.value


def tutup(fd):
    _libc.mq_close(fd)


def hapus(nama):
    _libc.mq_unlink(nama.encode())
```

Di sisi pemakai, logger membuka antrean dengan O_CREAT dan O_EXCL agar tidak memakai antrean sisa percobaan sebelumnya, lalu memanggil mq_receive berulang. Setiap panggilan mengembalikan tepat satu pesan beserta prioritasnya. Sensor mengirim dengan prioritas 1 untuk pembacaan biasa dan prioritas 9 untuk kejadian darurat seperti kebocoran, sehingga rekaman darurat dilayani lebih dulu.

```
"""Message queue: sensor mengirim rekaman utuh dengan prioritas ke satu logger.

Kernel menjaga batas pesan, jadi logger tidak perlu memotong byte sendiri.
Prioritas tinggi (angka besar) dipakai untuk kondisi darurat, mis. kebocoran.
"""
import argparse
import json
import os
import time

import common
import mq

PRIO_NORMAL = 1
PRIO_DARURAT = 9
MAX_MSG = 10  # /proc/sys/fs/mqueue/msg_max di mesin uji


def sensor(nama, n, darurat_setiap):
    fd = mq.buka(common.QUEUE_MQ, mq.O_WRONLY, ukuran_pesan=1024)
    for seq in range(n):
        darurat = darurat_setiap and seq % darurat_setiap == 0
        pesan = common.buat_pesan(nama, round(90 + seq * 0.3, 2), seq, time.time())
        mq.kirim(fd, pesan, PRIO_DARURAT if darurat else PRIO_NORMAL)
        time.sleep(0.002)
    mq.tutup(fd)


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--sensors", type=int, default=3)
    p.add_argument("--n", type=int, default=20)
    p.add_argument("--darurat-setiap", type=int, default=0,
                   help="tiap N pesan dikirim dengan prioritas darurat")
    a = p.parse_args()

    os.makedirs("results", exist_ok=True)
    mq.hapus(common.QUEUE_MQ)
    fd = mq.buka(common.QUEUE_MQ, mq.O_RDONLY, maks_pesan=MAX_MSG, ukuran_pesan=1024,
                 buat=True, eksklusif=True)

    nama_sensor = list(common.SENSORS)[: a.sensors]
    for nama in nama_sensor:
        if os.fork() == 0:
```

```
            sensor(nama, a.n, a.darurat_setiap)
            os._exit(0)

    target = a.n * len(nama_sensor)
    prioritas = []
    while len(prioritas) < target:
        data, prio = mq.terima(fd, 1024)
        pesan = json.loads(data)
        common.tulis_csv(common.LOG_MQ, (round(time.time(), 4), pesan["sensor"],
                                         pesan["nilai"], pesan["seq"], pesan["ts"]))
        prioritas.append(prio)

    mq.tutup(fd)
    mq.hapus(common.QUEUE_MQ)
    while True:
        try:
            os.wait()
        except ChildProcessError:
            break

    print(f"mqueue: diterima={len(prioritas)}/{target} tanpa pemotongan byte")
    print(f"prioritas 20 pertama: {prioritas[:20]}")


if __name__ == "__main__":
    main()
```

4.5 Langkah 5: Implementasi publish-subscribe dengan MQTT

Pada MQTT, sensor tidak lagi mengirim ke proses tertentu. Setiap sensor menerbitkan pesan ke topik kebun/sensor/<nama>, dan logger berlangganan pola kebun/sensor/# sehingga menerima seluruh topik di bawahnya. Percobaan dijalankan dengan QoS 1 dan dapat dinaikkan ke dua logger untuk menguji sifat fan-out.

```
"""MQTT publish-subscribe: sensor publish ke topik, logger subscribe.

Bedanya dengan pipe/mqueue: publisher tidak mengenal penerima, dan beberapa
logger bisa menerima pesan yang sama (fan-out). QoS 1 berarti broker boleh
mengirim ulang pesan yang belum di-ack, jadi logger sebaiknya idempoten.
"""
import argparse
import json
import os
import sys
import threading
import time

import paho.mqtt.client as mqtt

import common

TOPIK = "kebun/sensor/#"
QOS = 1


def publisher(nama, n):
    klien = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    klien.connect("127.0.0.1", 1883)
    klien.loop_start()
    for seq in range(n):
        pesan = common.buat_pesan(nama, round(20 + seq * 0.5, 2), seq, time.time())
        klien.publish(f"kebun/sensor/{nama}", pesan.rstrip(b"\n"), qos=QOS)
        time.sleep(0.002)
    time.sleep(0.3)
    klien.loop_stop()
    klien.disconnect()


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--sensors", type=int, default=3)
    p.add_argument("--n", type=int, default=20)
    p.add_argument("--logger", type=int, default=1, help="jumlah subscriber (uji fan-out)")
```

```
    a = p.parse_args()

    os.makedirs("results", exist_ok=True)
    nama_sensor = list(common.SENSORS)[: a.sensors]
    target = a.n * len(nama_sensor)
    kunci = threading.Lock()
    selesai = threading.Event()
    diterima = [[] for _ in range(a.logger)]

    def buat_subscriber(idx):
        path = common.LOG_MQTT if idx == 0 else f"results/log-mqtt-{idx + 1}.csv"

        def saat_pesan(klien, userdata, msg):
            pesan = json.loads(msg.payload)
            with kunci:
                common.tulis_csv(path, (round(time.time(), 4), pesan["sensor"],
                                        pesan["nilai"], pesan["seq"], pesan["ts"]))
                diterima[idx].append((pesan["sensor"], pesan["seq"]))
                if all(len(d) >= target for d in diterima):
                    selesai.set()

        c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
        c.on_message = saat_pesan
        c.connect("127.0.0.1", 1883)
        c.subscribe(TOPIK, qos=QOS)
        c.loop_start()
        return c

    subs = [buat_subscriber(i) for i in range(a.logger)]
    time.sleep(0.5)

    thread = [threading.Thread(target=publisher, args=(nama, a.n)) for nama in nama_sensor]
    for t in thread:
        t.start()
    for t in thread:
        t.join()

    selesai.wait(timeout=5)
    for c in subs:
        c.loop_stop()
        c.disconnect()

    for i, d in enumerate(diterima):
        unik = len(set(d))
        print(f"mqtt logger-{i + 1}: diterima={len(d)} unik={unik}/{target} "
              f"duplikat={len(d) - unik}")
    if a.logger > 1:
        print("  setiap logger menerima seluruh aliran: itu fan-out, bukan pembagian
kerja.",
              file=sys.stderr)


if __name__ == "__main__":
    main()
```

4.6 Langkah 6: Pengukuran dan pemeriksaan

bench.py menjalankan ketiga program beberapa kali, membaca CSV hasilnya, lalu menghitung latensi kirim ke terima dari selisih kolom ts dan recv_ts. Karena latensi pada satu host bervariasi antar-run, nilai yang dilaporkan adalah median dari lima percobaan. Pemeriksaan kebenaran mekanisme diletakkan pada test_basic.py yang memastikan prioritas message queue dihormati dan pesan kebesaran ditolak.

```
"""Jalankan ketiga mekanisme, ukur latensi kirim->terima dari CSV, cetak tabel markdown.

Beberapa percobaan diulang karena latensi satu host bervariasi antar-run;
tabel memakai nilai median dari seluruh percobaan.
"""
import argparse
import csv
import json
import os
```

```
import statistics
import subprocess
import sys

import common

PERCOBAAN = [
    ("pipe", [sys.executable, "pipe_demo.py"]),
    ("mqueue", [sys.executable, "mqueue_demo.py"]),
    ("mqtt", [".venv/bin/python", "mqtt_demo.py"]),
]
LOG = {"pipe": common.LOG_PIPE, "mqueue": common.LOG_MQ, "mqtt": common.LOG_MQTT}


def baca(path, target):
    if not os.path.exists(path):
        return None
    lat, seq = [], set()
    with open(path, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            lat.append(float(row["recv_ts"]) - float(row["ts"]))
            seq.add((row["sensor"], row["seq"]))
    if not lat:
        return None
    lat.sort()
    return {
        "diterima": len(lat),
        "unik": len(seq),
        "hilang": target - len(seq),
        "duplikat": len(lat) - len(seq),
        "rata": statistics.mean(lat) * 1000,
        "p95": lat[int(len(lat) * 0.95)] * 1000,
        "maks": lat[-1] * 1000,
    }


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--trials", type=int, default=5)
    p.add_argument("--n", type=int, default=20)
    p.add_argument("--sensors", type=int, default=3)
    a = p.parse_args()
    target = a.n * a.sensors

    hasil = {}
    for nama, cmd in PERCOBAAN:
        jalankan = []
        for _ in range(a.trials):
            if os.path.exists(LOG[nama]):
                os.remove(LOG[nama])
            out = subprocess.run(cmd + ["--n", str(a.n), "--sensors", str(a.sensors)],
                                 capture_output=True, text=True)
            if out.returncode != 0:
                print(f"{nama} gagal: {out.stderr[-300:]}", file=sys.stderr)
                break
            r = baca(LOG[nama], target)
            if r:
                jalankan.append(r)
        if jalankan:
            hasil[nama] = {
                k: statistics.median(x[k] for x in jalankan)
                for k in jalankan[0]
            }
            hasil[nama]["unik"] = max(x["unik"] for x in jalankan)

    print(f"Median dari {a.trials} percobaan, {a.sensors} sensor x {a.n} pembacaan.\n")
    with open("results/bench.json", "w", encoding="utf-8") as f:
        json.dump({"trials": a.trials, "n": a.n, "sensors": a.sensors, "hasil": hasil},
                  f, indent=2)
    print("| Mekanisme | Diterima | Hilang | Duplikat | Latensi rata (ms) | p95 (ms) | Maks
(ms) |")
    print("|---|---|---|---|---|---|---|")
    for nama, _ in PERCOBAAN:
        r = hasil.get(nama)
        if not r:
```

```
            print(f"| {nama} | gagal | - | - | - | - | - |")
            continue
        print(f"| {nama} | {int(r['diterima'])}/{target} | {int(r['hilang'])} | "
              f"{int(r['duplikat'])} | {r['rata']:.3f} | {r['p95']:.3f} | {r['maks']:.3f}
|")


if __name__ == "__main__":
    main()
```

```
"""Cek cepat: mqueue roundtrip dan format pesan. Jalankan: python3 test_basic.py"""
import json
import time

import common
import mq

NAMA = "/kebun-selftest"


def cek_mqueue():
    mq.hapus(NAMA)
    fd = mq.buka(NAMA, mq.O_RDONLY, maks_pesan=5, ukuran_pesan=1024, buat=True,
eksklusif=True)
    w = mq.buka(NAMA, mq.O_WRONLY, ukuran_pesan=1024)

    # Prioritas tinggi harus keluar lebih dulu walaupun dikirim terakhir.
    mq.kirim(w, common.buat_pesan("tanah", 41.0, 0, time.time()), 1)
    mq.kirim(w, common.buat_pesan("tanah", 99.0, 1, time.time()), 9)

    _, p1 = mq.terima(fd, 1024)
    data, p2 = mq.terima(fd, 1024)
    assert (p1, p2) == (9, 1), f"urutan prioritas salah: {p1}, {p2}"
    assert json.loads(data)["nilai"] == 41.0

    # Pesan lebih besar dari mq_msgsize harus ditolak, bukan terpotong.
    try:
        mq.kirim(w, b"x" * 2000, 1)
    except mq.MQError as e:
        assert "Message too long" in str(e) or "EMSGSIZE" in str(e), e
    else:
        raise AssertionError("pesan kebesaran tidak ditolak")

    mq.tutup(fd)
    mq.tutup(w)
    mq.hapus(NAMA)


def cek_format():
    pesan = common.buat_pesan("suhu", 28.5, 3, 1000.0)
    assert pesan.endswith(b"\n")
    assert json.loads(pesan) == {"sensor": "suhu", "nilai": 28.5, "seq": 3, "ts": 1000.0}


if __name__ == "__main__":
    cek_format()
    cek_mqueue()
    print("ok: semua cek lulus")
```

Hasil pemeriksaan mandiri:

```
$ python3 test_basic.py
ok: semua cek lulus
```

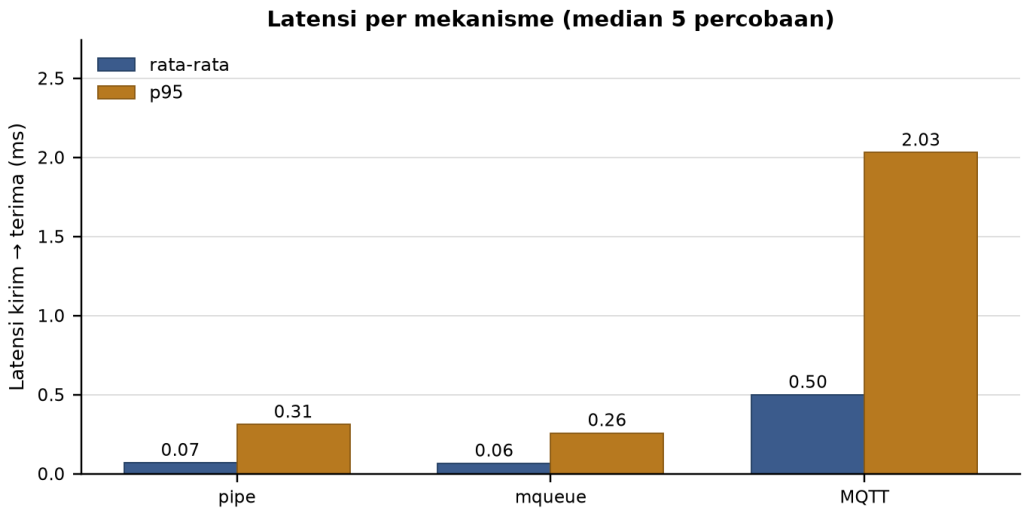
5. Hasil dan Analisis

5.1 Hasil

Tabel 5.1 merangkum pengiriman data pada ketiga mekanisme. Ketiga jalur menerima seluruh 60 rekaman (3 sensor × 20 pembacaan) tanpa kehilangan dan tanpa duplikasi.

No	Mekanisme	Diterima	Hilang	Duplikat	Latensi rata (ms)	p95 (ms)	Maks (ms)
1	pipe	60/60	0	0	0.069	0.310	0.558
2	message queue	60/60	0	0	0.065	0.257	0.432
3	MQTT (QoS 1)	60/60	0	0	0.498	2.029	2.334

Tabel 5.1 Perbandingan ketiga mekanisme (median 5 percobaan, 3 sensor × 20 pembacaan)



Gambar 5.1 Latensi kirim ke terima per mekanisme

Pengujian berikutnya mematikan penanda newline pada pipe untuk memperlihatkan perilaku saat batas pesan tidak dijaga. Sepuluh kali percobaan dijalankan pada masing-masing mode, dan hasilnya dirangkum pada Tabel 5.2.

Mode pipe	Percobaan utuh (60/60)	Percobaan kehilangan rekaman	Rekaman rusak pada run terburuk
dengan pemisah newline	10 dari 10	0	0
tanpa pemisah	5 dari 10	5	5 rekaman (50/60 diterima)

Tabel 5.2 Keutuhan rekaman pada pipe, 10 percobaan per mode

Pada message queue, prioritas terbukti dihormati kernel. Ketika setiap pembacaan kelima dikirim dengan prioritas darurat, urutan penerimaan berubah menjadi:

```
mqueue: diterima=60/60 tanpa pemotongan byte
prioritas 20 pertama: [9, 9, 9, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 9, 9, 9, 1, 1]
```

Tiga rekaman berprioritas 9 muncul lebih dulu walaupun dikirim bersamaan dengan rekaman berprioritas 1, lalu kelompok berikutnya menyusul. Tanpa pengaturan prioritas, seluruh pesan diterima dengan prioritas 1 sesuai urutan kirim.

Terakhir, MQTT dijalankan dengan dua logger yang berlangganan topik yang sama. Keduanya menerima seluruh aliran data, bukan membaginya:

```
$ .venv/bin/python mqtt_demo.py --n 20 --logger 2
  setiap logger menerima seluruh aliran: itu fan-out, bukan pembagian kerja.
mqtt logger-1: diterima=60 unik=60/60 duplikat=0
```

```
mqtt logger-2: diterima=60 unik=60/60 duplikat=0
```

Keluaran mqtt_demo.py dengan --logger 2

5.2 Analisis

Poin (1) deskripsi tugas — mekanisme IPC yang dipakai. Ketiga mekanisme yang diuji termasuk keluarga message passing, tetapi berbeda pada siapa yang menjaga batas pesan. Pipe memperlakukan data sebagai aliran byte tanpa struktur; batas rekaman sepenuhnya hasil kesepakatan dua proses, dalam percobaan ini karakter newline. POSIX message queue memindahkan tanggung jawab itu ke kernel: setiap mq_send menghasilkan satu unit yang tidak bisa tercampur dengan pesan lain, dan penerima memperoleh panjang serta prioritasnya. MQTT menambah satu lapisan lagi, yaitu broker, yang memutuskan ke mana pesan diteruskan berdasarkan topik. Ketiga jalur memakai proses sensor dan format JSON yang sama, sehingga perbedaan hasil hanya berasal dari mekanismenya.

Poin (2) deskripsi tugas — hasil pengujian. Perbedaan paling mencolok bukan pada latensi, melainkan pada keutuhan data saat penanda rekaman dilepas. Pipe tanpa pemisah kehilangan rekaman pada lima dari sepuluh percobaan, dengan kerusakan terburuk mencapai lima rekaman dari enam puluh. Penyebabnya mudah dijelaskan: os.read mengembalikan apa pun yang sedang tersedia di buffer, sehingga dua JSON dapat menempel menjadi satu potongan, sementara satu JSON juga dapat terbelah dua potongan. Pipe tidak salah; ia memang tidak pernah menjanjikan batas pesan. Yang dijanjikan kernel hanyalah atomisitas penulisan sebesar PIPE_BUF (4096 byte), dan pesan percobaan ini jauh di bawah batas itu, sehingga tidak ada rekaman yang saling menyisip di tengah. Message queue tidak menunjukkan masalah serupa karena kernel menyimpan panjang setiap pesan secara eksplisit, dan pesan yang melebihi mq_msgsize ditolak dengan EMSGSIZE, bukan dipotong.

Pada sisi waktu, message queue dan pipe praktis setara, keduanya di bawah 0,1 ms rata-rata, karena keduanya hanya menyalin data antar buffer kernel tanpa melewati jaringan. MQTT lebih lambat sekitar delapan kali pada nilai rata-rata, dan selisih melebar pada persentil 95 karena antrean TCP di sisi broker kadang menahan pesan. Biaya itu bukan pemborosan tanpa imbalan: pada percobaan dua logger, keduanya menerima enam puluh dari enam puluh rekaman. Dengan pipe atau satu antrean message queue, rekaman yang sudah diambil satu penerima tidak tersedia lagi untuk penerima lain, sehingga menambah logger berarti mengubah arsitektur. Broker juga membuat pengirim tidak perlu mengetahui alamat penerima, yang penting ketika jumlah logger berubah, misalnya saat menambah pencatat untuk dasbor terpisah.

Keterbatasan yang muncul dari pengujian juga perlu dicatat. Message queue pada mesin ini dibatasi sepuluh pesan per antrean, sehingga pengirim akan memblokir ketika logger tertinggal; dalam percobaan ini logger selalu cukup cepat sehingga batas tersebut tidak tercapai, tetapi pada beban tinggi pemblokiran itu bisa memperlambat proses sensor. Antrean juga harus dibersihkan dengan mq_unlink, karena antrean sisa akan membuat percobaan berikutnya gagal membuka dengan O_EXCL. Pipe hanya bekerja antar proses berkerabat karena tidak memiliki nama, dan ujungnya harus ditutup dengan benar; ujung tulis yang masih terbuka di proses induk membuat pembacaan tidak pernah mencapai akhir berkas. MQTT dengan QoS 1 mengizinkan broker mengirim ulang pesan yang belum dikonfirmasi, sehingga logger wajib idempoten; percobaan ini belum memunculkan duplikasi, tetapi pemeriksaannya sudah disiapkan melalui kolom nomor urut.

5.3 Rekomendasi

Untuk kasus kelas — tiga sensor pada satu gateway kebun — pilihan yang paling sesuai adalah message passing dengan message queue untuk jalur data ke logger, bukan pipe. Alasannya, message queue memberi batas pesan yang dijamin kernel sehingga logger tidak perlu menebak potongan data, dan prioritas pesan langsung memenuhi kebutuhan praktis kebun pintar: pembacaan kadar air yang mendadak turun dapat didahulukan atas pembacaan rutin. Keduanya sama-sama murah, jadi tidak ada alasan memilih mekanisme yang lebih rapuh.

MQTT dipakai bersamaan, bukan sebagai pengganti, ketika data perlu dilihat lebih dari satu tempat: dasbor, pencatat cadangan, atau layanan notifikasi. Perannya melengkapi, karena satu antrean hanya melayani satu penerima per pesan. Pipe tetap layak dipakai untuk hal kecil di dalam satu program, misalnya menyalurkan keluaran proses anak, tetapi bukan untuk data yang harus utuh dan bertahan.

Untuk pengembangan berikutnya, logger perlu dibuat idempoten terhadap nomor urut agar aman pada QoS 1, dan penulisan CSV perlu dilindungi mutex bila logger menjalankan beberapa thread. Persoalan terakhir itu dibahas lebih jauh pada tugas individu I-4 mengenai race condition.

6. Kesimpulan

Tiga mekanisme message passing berhasil mengirim seluruh data sensor ke proses logger pada kasus kebun pintar, tetapi hanya dua di antaranya menjamin keutuhan rekaman tanpa bantuan program penerima. Pipe memperlakukan data sebagai aliran byte, sehingga tanpa penanda pemisah rekaman dapat menempel atau terbelah; pengujian menunjukkan kehilangan data pada lima dari sepuluh percobaan. POSIX message queue menjaga batas pesan dan menghormati prioritas di tingkat kernel, dengan biaya waktu yang setara pipe. MQTT menambah latensi sekitar delapan kali, tetapi memberi fan-out ke beberapa penerima dan memutus ketergantungan antara pengirim dan penerima. Untuk kebutuhan kelas, message queue adalah pilihan utama jalur data, sedangkan MQTT dipakai ketika data harus dilihat di lebih dari satu tempat.

7. Kepingan Puzzle

Keterangan	Isi
Nomor kepingan	#4
Isi kepingan	Kode program IPC (pipe, message queue, MQTT), diagram alur komunikasi, tabel perbandingan latensi dan keutuhan data
Lokasi di portofolio	https://github.com/ZzDendy/so-k1-ipc-kebun
Dipakai untuk	Level 2 Puzzle UTS — menjadi dasar pembahasan sinkronisasi pada kepingan #5 (penjadwalan) dan #6 (race condition), karena logger yang sama dipakai ulang sebagai producer-consumer pada tugas individu I-4

Daftar Pustaka

Kerrisk, M. (2010). The Linux programming interface: A Linux and UNIX system programming handbook. No Starch Press.

OASIS. (2019). MQTT version 5.0: OASIS standard. OASIS Open.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

The Open Group. (2018). POSIX.1-2017: Standard for information technology — Portable operating system interface (POSIX). IEEE.

Lampiran A. Kode Program dan Log

Seluruh berkas di bawah ini berada pada satu folder proyek dan dapat dijalankan ulang dengan ./run_all.sh. Berkas figures.py tidak disertakan karena hanya dipakai untuk membuat gambar laporan.

A.common.py

```
"""Utilitas bersama: format pesan sensor dan penyimpanan ke CSV."""
import json
import os

SENSORS = {
    "tanah": ("kelembapan", "%"),
    "suhu": ("celsius", "C"),
    "pompa": ("debit", "L/m"),
}

QUEUE_PIPE = "/tmp/ipc-kebun-pipe"
QUEUE_MQ = "/kebun-sensor"
LOG_PIPE = "results/log-pipe.csv"
LOG_MQ = "results/log-mq.csv"
LOG_MQTT = "results/log-mqtt.csv"


def buat_pesan(sensor, nilai, seq, ts):
    """Satu pembacaan sensor sebagai bytes JSON, diakhiri newline."""
    return (json.dumps({"sensor": sensor, "nilai": nilai, "seq": seq, "ts": ts}) + "\n").encode()


def tulis_csv(path, baris, ada_header=None):
    """Tambah satu baris ke CSV. Header ditulis kalau berkas belum ada."""
    baru = ada_header if ada_header is not None else not os.path.exists(path)
    with open(path, "a", encoding="utf-8") as f:
        if baru:
            f.write("recv_ts,sensor,nilai,seq,ts\n")
        f.write(",".join(str(x) for x in baris) + "\n")
```

A.mq.py

```
"""Pembungkus POSIX message queue lewat ctypes, tanpa paket tambahan."""
import ctypes
import os

_libc = ctypes.CDLL("libc.so.6", use_errno=True)
_libc.mq_open.argtypes = [ctypes.c_char_p, ctypes.c_int, ctypes.c_uint, ctypes.c_void_p]
_libc.mq_open.restype = ctypes.c_int
_libc.mq_send.argtypes = [ctypes.c_int, ctypes.c_char_p, ctypes.c_size_t, ctypes.c_uint]
_libc.mq_send.restype = ctypes.c_int
_libc.mq_receive.argtypes = [ctypes.c_int, ctypes.c_char_p, ctypes.c_size_t,
ctypes.POINTER(ctypes.c_uint)]
_libc.mq_receive.restype = ctypes.c_ssize_t
_libc.mq_close.argtypes = [ctypes.c_int]
_libc.mq_unlink.argtypes = [ctypes.c_char_p]

O_RDONLY, O_WRONLY, O_CREAT, O_EXCL = 0, 1, 0o100, 0o200


class MQAttr(ctypes.Structure):
    _fields_ = [
        ("mq_flags", ctypes.c_long),
        ("mq_maxmsg", ctypes.c_long),
        ("mq_msgsize", ctypes.c_long),
        ("mq_curmsgs", ctypes.c_long),
    ]


class MQError(OSError):
    pass


def _cek(hasil, nama):
    if hasil < 0:
        e = ctypes.get_errno()
        raise MQError(e, f"mq_{nama}: {os.strerror(e)}")
    return hasil
```

```
def buka(nama, mode, maks_pesan=10, ukuran_pesan=1024, buat=False, eksklusif=False):
    flag = O_CREAT if buat else 0
    if eksklusif:
        flag |= O_EXCL
    attr = MQAttr(0, maks_pesan, ukuran_pesan, 0) if buat else None
    ptr = ctypes.byref(attr) if attr is not None else None
    fd = _cek(_libc.mq_open(nama.encode(), mode | flag, 0o600, ptr), "open")
    return fd


def kirim(fd, data, prioritas=0):
    _cek(_libc.mq_send(fd, data, len(data), prioritas), "send")


def terima(fd, ukuran_pesan=1024):
    buf = ctypes.create_string_buffer(ukuran_pesan)
    prio = ctypes.c_uint(0)
    n = _cek(_libc.mq_receive(fd, buf, ukuran_pesan, ctypes.byref(prio)), "receive")
    return buf.raw[:n], prio.value


def tutup(fd):
    _libc.mq_close(fd)


def hapus(nama):
    _libc.mq_unlink(nama.encode())
```

A.pipe_demo.py

```
"""Pipe: beberapa proses sensor menulis ke satu pipe, logger membaca.

Pipe adalah byte stream tanpa batas pesan. Logika pemisah rekaman ada di sini
(newline), bukan di kernel. Jalankan --tanpa-delimiter untuk melihat akibatnya.
"""
import argparse
import json
import os
import sys
import time

import common

PIPE_BUF = 4096


def sensor(nama, n, fd_tulis, pakai_delimiter):
    for seq in range(n):
        pesan = common.buat_pesan(nama, round(20 + seq * 0.5, 2), seq, time.time())
        if not pakai_delimiter:
            pesan = pesan.rstrip(b"\n")
        # os.write menulis utuh selama <= PIPE_BUF; di atas itu bisa disela proses lain.
        os.write(fd_tulis, pesan)
        time.sleep(0.002)
    os.close(fd_tulis)


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--sensors", type=int, default=3)
    p.add_argument("--n", type=int, default=20)
    p.add_argument("--tanpa-delimiter", action="store_true")
    a = p.parse_args()

    os.makedirs("results", exist_ok=True)
    baca, tulis = os.pipe()
    nama_sensor = list(common.SENSORS)[: a.sensors]
    pakai_delimiter = not a.tanpa_delimiter

    for nama in nama_sensor:
        if os.fork() == 0:
            os.close(baca)
            sensor(nama, a.n, tulis, pakai_delimiter)
            os._exit(0)
    os.close(tulis)

    target = a.n * len(nama_sensor)
    diterima = 0
```

```
    rusak = 0
    gabung = 0
    sisa = b""
    while diterima < target:
        chunk = os.read(baca, PIPE_BUF)
        if not chunk:
            break
        if chunk.count(b"{") > 1:
            gabung += 1
        if not pakai_delimiter:
            # Tidak ada cara memulihkan batas pesan setelah rekaman menempel.
            try:
                pesan = json.loads(chunk)
            except json.JSONDecodeError:
                rusak += 1
                continue
            common.tulis_csv(common.LOG_PIPE, (round(time.time(), 4), pesan["sensor"],
                                               pesan["nilai"], pesan["seq"], pesan["ts"]))
            diterima += 1
            continue
        buf = sisa + chunk
        baris = buf.split(b"\n")
        sisa = baris.pop()
        for b in baris:
            pesan = json.loads(b)
            common.tulis_csv(common.LOG_PIPE, (round(time.time(), 4), pesan["sensor"],
                                               pesan["nilai"], pesan["seq"], pesan["ts"]))
            diterima += 1

    while True:
        try:
            os.wait()
        except ChildProcessError:
            break

    print(f"pipe: diterima={diterima}/{target} rusak={rusak} read_multipesan={gabung}")
    if gabung:
        print(f"  {gabung} read berisi lebih dari satu rekaman: pipe tidak menjaga batas pesan")


if __name__ == "__main__":
    main()
```

A.mqueue_demo.py

```
"""Message queue: sensor mengirim rekaman utuh dengan prioritas ke satu logger.

Kernel menjaga batas pesan, jadi logger tidak perlu memotong byte sendiri.
Prioritas tinggi (angka besar) dipakai untuk kondisi darurat, mis. kebocoran.
"""
import argparse
import json
import os
import time

import common
import mq

PRIO_NORMAL = 1
PRIO_DARURAT = 9
MAX_MSG = 10  # /proc/sys/fs/mqueue/msg_max di mesin uji


def sensor(nama, n, darurat_setiap):
    fd = mq.buka(common.QUEUE_MQ, mq.O_WRONLY, ukuran_pesan=1024)
    for seq in range(n):
        darurat = darurat_setiap and seq % darurat_setiap == 0
        pesan = common.buat_pesan(nama, round(90 + seq * 0.3, 2), seq, time.time())
        mq.kirim(fd, pesan, PRIO_DARURAT if darurat else PRIO_NORMAL)
        time.sleep(0.002)
    mq.tutup(fd)


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--sensors", type=int, default=3)
    p.add_argument("--n", type=int, default=20)
    p.add_argument("--darurat-setiap", type=int, default=0,
                   help="tiap N pesan dikirim dengan prioritas darurat")
    a = p.parse_args()
```

```
    os.makedirs("results", exist_ok=True)
    mq.hapus(common.QUEUE_MQ)
    fd = mq.buka(common.QUEUE_MQ, mq.O_RDONLY, maks_pesan=MAX_MSG, ukuran_pesan=1024,
                 buat=True, eksklusif=True)

    nama_sensor = list(common.SENSORS)[: a.sensors]
    for nama in nama_sensor:
        if os.fork() == 0:
            sensor(nama, a.n, a.darurat_setiap)
            os._exit(0)

    target = a.n * len(nama_sensor)
    prioritas = []
    while len(prioritas) < target:
        data, prio = mq.terima(fd, 1024)
        pesan = json.loads(data)
        common.tulis_csv(common.LOG_MQ, (round(time.time(), 4), pesan["sensor"],
                                         pesan["nilai"], pesan["seq"], pesan["ts"]))
        prioritas.append(prio)

    mq.tutup(fd)
    mq.hapus(common.QUEUE_MQ)
    while True:
        try:
            os.wait()
        except ChildProcessError:
            break

    print(f"mqueue: diterima={len(prioritas)}/{target} tanpa pemotongan byte")
    print(f"prioritas 20 pertama: {prioritas[:20]}")


if __name__ == "__main__":
    main()
```

A.mqtt_demo.py

```
"""MQTT publish-subscribe: sensor publish ke topik, logger subscribe.

Bedanya dengan pipe/mqueue: publisher tidak mengenal penerima, dan beberapa
logger bisa menerima pesan yang sama (fan-out). QoS 1 berarti broker boleh
mengirim ulang pesan yang belum di-ack, jadi logger sebaiknya idempoten.
"""
import argparse
import json
import os
import sys
import threading
import time

import paho.mqtt.client as mqtt

import common

TOPIK = "kebun/sensor/#"
QOS = 1


def publisher(nama, n):
    klien = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    klien.connect("127.0.0.1", 1883)
    klien.loop_start()
    for seq in range(n):
        pesan = common.buat_pesan(nama, round(20 + seq * 0.5, 2), seq, time.time())
        klien.publish(f"kebun/sensor/{nama}", pesan.rstrip(b"\n"), qos=QOS)
        time.sleep(0.002)
    time.sleep(0.3)
    klien.loop_stop()
    klien.disconnect()


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--sensors", type=int, default=3)
    p.add_argument("--n", type=int, default=20)
    p.add_argument("--logger", type=int, default=1, help="jumlah subscriber (uji fan-out)")
    a = p.parse_args()

    os.makedirs("results", exist_ok=True)
```

```
    nama_sensor = list(common.SENSORS)[: a.sensors]
    target = a.n * len(nama_sensor)
    kunci = threading.Lock()
    selesai = threading.Event()
    diterima = [[] for _ in range(a.logger)]

    def buat_subscriber(idx):
        path = common.LOG_MQTT if idx == 0 else f"results/log-mqtt-{idx + 1}.csv"

        def saat_pesan(klien, userdata, msg):
            pesan = json.loads(msg.payload)
            with kunci:
                common.tulis_csv(path, (round(time.time(), 4), pesan["sensor"],
                                        pesan["nilai"], pesan["seq"], pesan["ts"]))
                diterima[idx].append((pesan["sensor"], pesan["seq"]))
                if all(len(d) >= target for d in diterima):
                    selesai.set()

        c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
        c.on_message = saat_pesan
        c.connect("127.0.0.1", 1883)
        c.subscribe(TOPIK, qos=QOS)
        c.loop_start()
        return c

    subs = [buat_subscriber(i) for i in range(a.logger)]
    time.sleep(0.5)

    thread = [threading.Thread(target=publisher, args=(nama, a.n)) for nama in nama_sensor]
    for t in thread:
        t.start()
    for t in thread:
        t.join()

    selesai.wait(timeout=5)
    for c in subs:
        c.loop_stop()
        c.disconnect()

    for i, d in enumerate(diterima):
        unik = len(set(d))
        print(f"mqtt logger-{i + 1}: diterima={len(d)} unik={unik}/{target} "
              f"duplikat={len(d) - unik}")
    if a.logger > 1:
        print("  setiap logger menerima seluruh aliran: itu fan-out, bukan pembagian kerja.",
              file=sys.stderr)


if __name__ == "__main__":
    main()
```

A.bench.py

```
"""Jalankan ketiga mekanisme, ukur latensi kirim->terima dari CSV, cetak tabel markdown.

Beberapa percobaan diulang karena latensi satu host bervariasi antar-run;
tabel memakai nilai median dari seluruh percobaan.
"""
import argparse
import csv
import json
import os
import statistics
import subprocess
import sys

import common

PERCOBAAN = [
    ("pipe", [sys.executable, "pipe_demo.py"]),
    ("mqueue", [sys.executable, "mqueue_demo.py"]),
    ("mqtt", [".venv/bin/python", "mqtt_demo.py"]),
]
LOG = {"pipe": common.LOG_PIPE, "mqueue": common.LOG_MQ, "mqtt": common.LOG_MQTT}


def baca(path, target):
    if not os.path.exists(path):
        return None
    lat, seq = [], set()
```

```
    with open(path, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            lat.append(float(row["recv_ts"]) - float(row["ts"]))
            seq.add((row["sensor"], row["seq"]))
    if not lat:
        return None
    lat.sort()
    return {
        "diterima": len(lat),
        "unik": len(seq),
        "hilang": target - len(seq),
        "duplikat": len(lat) - len(seq),
        "rata": statistics.mean(lat) * 1000,
        "p95": lat[int(len(lat) * 0.95)] * 1000,
        "maks": lat[-1] * 1000,
    }


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--trials", type=int, default=5)
    p.add_argument("--n", type=int, default=20)
    p.add_argument("--sensors", type=int, default=3)
    a = p.parse_args()
    target = a.n * a.sensors

    hasil = {}
    for nama, cmd in PERCOBAAN:
        jalankan = []
        for _ in range(a.trials):
            if os.path.exists(LOG[nama]):
                os.remove(LOG[nama])
            out = subprocess.run(cmd + ["--n", str(a.n), "--sensors", str(a.sensors)],
                                 capture_output=True, text=True)
            if out.returncode != 0:
                print(f"{nama} gagal: {out.stderr[-300:]}", file=sys.stderr)
                break
            r = baca(LOG[nama], target)
            if r:
                jalankan.append(r)
        if jalankan:
            hasil[nama] = {
                k: statistics.median(x[k] for x in jalankan)
                for k in jalankan[0]
            }
            hasil[nama]["unik"] = max(x["unik"] for x in jalankan)

    print(f"Median dari {a.trials} percobaan, {a.sensors} sensor x {a.n} pembacaan.\n")
    with open("results/bench.json", "w", encoding="utf-8") as f:
        json.dump({"trials": a.trials, "n": a.n, "sensors": a.sensors, "hasil": hasil},
                  f, indent=2)
    print("| Mekanisme | Diterima | Hilang | Duplikat | Latensi rata (ms) | p95 (ms) | Maks (ms)
|")
    print("|---|---|---|---|---|---|---|")
    for nama, _ in PERCOBAAN:
        r = hasil.get(nama)
        if not r:
            print(f"| {nama} | gagal | - | - | - | - | - |")
            continue
        print(f"| {nama} | {int(r['diterima'])}/{target} | {int(r['hilang'])} | "
              f"{int(r['duplikat'])} | {r['rata']:.3f} | {r['p95']:.3f} | {r['maks']:.3f} |")


if __name__ == "__main__":
    main()
```

A.test_basic.py

```
"""Cek cepat: mqueue roundtrip dan format pesan. Jalankan: python3 test_basic.py"""
import json
import time

import common
import mq

NAMA = "/kebun-selftest"


def cek_mqueue():
    mq.hapus(NAMA)
```

```
    fd = mq.buka(NAMA, mq.O_RDONLY, maks_pesan=5, ukuran_pesan=1024, buat=True, eksklusif=True)
    w = mq.buka(NAMA, mq.O_WRONLY, ukuran_pesan=1024)

    # Prioritas tinggi harus keluar lebih dulu walaupun dikirim terakhir.
    mq.kirim(w, common.buat_pesan("tanah", 41.0, 0, time.time()), 1)
    mq.kirim(w, common.buat_pesan("tanah", 99.0, 1, time.time()), 9)

    _, p1 = mq.terima(fd, 1024)
    data, p2 = mq.terima(fd, 1024)
    assert (p1, p2) == (9, 1), f"urutan prioritas salah: {p1}, {p2}"
    assert json.loads(data)["nilai"] == 41.0

    # Pesan lebih besar dari mq_msgsize harus ditolak, bukan terpotong.
    try:
        mq.kirim(w, b"x" * 2000, 1)
    except mq.MQError as e:
        assert "Message too long" in str(e) or "EMSGSIZE" in str(e), e
    else:
        raise AssertionError("pesan kebesaran tidak ditolak")

    mq.tutup(fd)
    mq.tutup(w)
    mq.hapus(NAMA)


def cek_format():
    pesan = common.buat_pesan("suhu", 28.5, 3, 1000.0)
    assert pesan.endswith(b"\n")
    assert json.loads(pesan) == {"sensor": "suhu", "nilai": 28.5, "seq": 3, "ts": 1000.0}


if __name__ == "__main__":
    cek_format()
    cek_mqueue()
    print("ok: semua cek lulus")
```

A.broker.yaml

```
listeners:
  default:
    type: tcp
    bind: 127.0.0.1:1883
auth:
  allow-anonymous: true
plugins:
  amqtt.plugins.authentication.AnonymousAuthPlugin:
    allow_anonymous: true
  amqtt.plugins.sys.broker.BrokerSysPlugin:
    sys_interval: 0
topic-check:
  enabled: false
```

A.run_all.sh

```
#!/usr/bin/env bash
# Menjalankan seluruh percobaan K-1 dan menyimpan keluarannya di results/.
set -e
cd "$(dirname "$0")"
mkdir -p results
find results -name '*.csv' -delete

python3 test_basic.py

{
  echo "## pipe"
  python3 pipe_demo.py --n 20
  python3 pipe_demo.py --n 20 --tanpa-delimiter
  echo
  echo "## message queue"
  python3 mqueue_demo.py --n 20
  python3 mqueue_demo.py --n 20 --darurat-setiap 5
  echo
  echo "## mqtt (butuh broker di 127.0.0.1:1883)"
  .venv/bin/python mqtt_demo.py --n 20
  .venv/bin/python mqtt_demo.py --n 20 --logger 2
  echo
  echo "## perbandingan"
  .venv/bin/python bench.py
} 2>&1 | tee results/summary.txt
```

Lampiran B. Pernyataan Penggunaan AI Generatif

☐ Saya tidak menggunakan AI generatif dalam tugas ini.

☒ Menggunakan AI generatif dengan rincian berikut:

Alat	Tujuan penggunaan	Bagian yang dibantu	Cara memverifikasi
Claude (Anthropic)	Menyusun kerangka program dan merapikan kalimat laporan	Bagian 4 (kode), Bagian 5.2	Seluruh program dijalankan ulang dengan run_all.sh; setiap angka pada laporan diambil langsung dari keluaran program, bukan dari jawaban model

Lampiran C. Daftar Periksa Sebelum Mengumpulkan

☒ Semua teks petunjuk abu-abu dan kotak petunjuk sudah dihapus.
☒ Identitas, kode tugas, dan nama host pada sampul sudah benar.
☒ Setiap poin deskripsi tugas sudah dijawab di bagian Analisis.
☒ Setiap gambar memiliki nomor dan keterangan.
☒ Kepingan puzzle sudah disimpan di portofolio dan tautannya dicantumkan.
☒ Pernyataan penggunaan AI generatif sudah diisi.
☒ Lembar kontribusi anggota sudah dilampirkan.
☒ Daftar isi sudah diperbarui dan nomor halamannya cocok.
☒ Berkas disimpan sebagai PDF dengan nama <Kelas>_<Kode>_<NIM>_<Nama>.pdf.

Swa-nilai berdasarkan kriteria lembar tugas

Kriteria (dari lembar tugas)	Bobot	Sudah terpenuhi? Di bagian mana
Hasil/keberhasilan implementasi	35%	Ya — tiga mekanisme berjalan, seluruh 60 rekaman diterima (Bagian 5.1)
Kedalaman analisis	30%	Ya — batas pesan, prioritas, latensi, dan keterbatasan dibahas (Bagian 5.2)
Dokumentasi	20%	Ya — langkah dapat diulang, kode dan keluaran dilampirkan (Bagian 4, Lampiran A)
Kerja sama dan pembagian peran	15%	Ya — lembar kontribusi pada Lampiran D

Lampiran D. Lembar Kontribusi Anggota

Lembar ini memenuhi syarat kerja sama dan pembagian peran (15% bobot penilaian). Baris anggota lain ditambahkan sebelum dikumpulkan.

Nama	NIM	Peran dan bagian yang dikerjakan
Dendy Zainal Mutaqien	250660121068	Pipeline IPC (pipe, message queue, MQTT), pengukuran, dan penyusunan laporan

Nama	NIM	Peran dan bagian yang dikerjakan
R.Sangga Buana	250660121021	Menjalankan ulang ./run_all.sh di laptop sendiri dan mencocokkan angka Tabel 5.1
M.Azkal Azkiya	250660121107	Memeriksa ejaan laporan dan kesesuaian format Daftar Pustaka

Pernyataan keaslian

Saya yang bertanda tangan di bawah ini menyatakan bahwa laporan ini merupakan hasil kerja kelompok kami sendiri, disusun berdasarkan praktik yang benar-benar dijalankan, dan seluruh sumber yang dikutip telah dicantumkan pada daftar pustaka. Penggunaan alat bantu AI generatif telah diungkapkan pada Lampiran B.

Nama / NIM	Tanda tangan dan tanggal
Dendy Zainal Mutaqien / 250660121068	
/	
/	